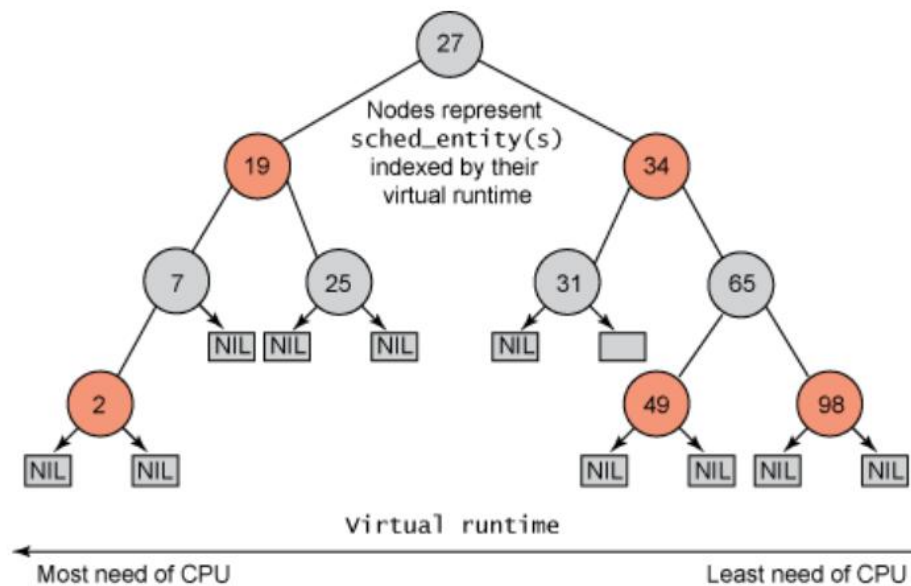


Red-Black Tree

A Red-Black Tree is a self-balancing binary search tree in which each node is assigned an additional property: a color, either red or black. This structure's primary purpose is to maintain balance during insertions and deletions, ensuring efficient operations for data retrieval and manipulation.

Properties of Red-Black Trees

1. Each node is either red or black.
2. The root of the tree is always black.
3. Red nodes cannot have red children (No parent-child colour should be red-red)
4. Every path from a node to its descendant null nodes (leaves) has the same number of black nodes.
5. All leaves (NIL nodes) are black.



For CFS, It needs a data structure which is

1. **Sorted** Data Structure
2. **Quick access** to the lowest runtime process as because that is the next in line to be executed
3. Efficient **Insertion** and **Deletion** because process constantly comes and go.

Therefore, CFS utilizes Red Black Tree.

Completely fair Scheduler (CFS)

Completely fair Scheduler (CFS) is a process schedulers currently used in Linux.

Process Scheduling -

As any program is loaded as process in RAM and then CPU executes the process according to the priority of the process.

1. Completely fair Scheduler (CFS) :

As the name suggests it fairly or equally divides the CPU time among all the processes. Before understanding the CFS let's look at the

Ideal Fair Scheduling (IFS) of N processes. If there are N processes in the ready queue then each process receives $(100/N)\%$ of CPU time according to IFS.

Let's take four process and their burst time as shown below waiting in the ready queue for the execution.

Process	Burst Time (in ms)
A	10
B	6
C	14
D	6

Take a time quantum of say 4ms. Initially, there is four processes waiting in the ready queue to be executed and according to Ideal fair scheduling, each process gets equally fair time for it's execution ($\text{Time quantum}/N$).

So $4/4=1$ each process gets 1ms to execute in first quantum. After the completion of six quantum process B and D are completely executed and remaining are A and C, which are already executed for 6ms and their remaining time is A=4ms and C=8ms).

In the seventh quantum of time A and C will execute ($4/2=2\text{ms}$ as there are only

two process remaining). This is ideal fair scheduling in which each process gets equally share of time quantum no matter what priority it is of.

Below is table description of the IFS.

Q:represent time quantum which is 4ms.

Process	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
A	1	1	1	1	1	1	2	2	-
B	1	1	1	1	1	1	-	-	-
C	1	1	1	1	1	1	2	2	4
D	1	1	1	1	1	1	-	-	-

CFS is similar as ideal-based scheduling instead it priorities each process according to their virtual runnable time.

Completely Fair Scheduler is a **preemptive process scheduler** introduced in **Linux kernel 2.6.23 (Current 6.13.12)**. Unlike traditional schedulers that use priorities or fixed time slices, **CFS aims to allocate CPU time as fairly as possible** to all tasks.

Key Concepts of CFS:

Concept	Description
Virtual Runtime (vruntime)	A key metric CFS uses to track how much CPU time a process has received.
Red-Black Tree	All runnable tasks are kept in a balanced Red-Black Tree sorted by their vruntime.
Fairness	The process with the lowest vruntime (i.e., least CPU usage so far) gets scheduled next.

Concept	Description
Weighting	CFS supports nice values. A "nicer" task gets less CPU; a less nice (or higher priority) task gets more.

Real-World Analogy:

Imagine a queue of people (processes) waiting to talk to a teacher (CPU). Each person gets a fair amount of time, but if someone has already talked a lot, the teacher picks someone who has talked less. That's CFS!

◇ Advantages of CFS

- Fair CPU sharing among processes.
- No starvation (unlike strict priority-based schedulers).
- Nice value support for weighted fairness.

◇ Limitations

- Not real-time: Doesn't guarantee deadlines.
 - Complexity of implementation (Red-Black Trees).
 - Estimating fairness in all scenarios is hard.
-

Let's now **extend our CFS example to include *priorities***, which in Linux are handled using **nice values**. In CFS, **priority affects the weight of each process**, and that affects **how fast the virtual runtime (vruntime) increases**.

How Priority Works in CFS

- CFS uses **vruntime** to decide who runs next.
- **Lower vruntime → higher chance to run.**
- But **vruntime increases faster for lower-priority processes**, and slower for higher-priority ones.
- This is how **fairness is weighted** by priority.

Nice Value	Weight (simplified)	Effect
-5	High priority	vruntime increases slowly
0	Normal priority	Normal vruntime rate
+5	Low priority	vruntime increases fast

For this example, let's simplify weights:

- **P1 (Nice = 0):** normal priority → weight = 1
- **P2 (Nice = -5):** high priority → weight = 0.5 (runs longer)
- **P3 (Nice = +5):** low priority → weight = 2 (runs shorter)

♦ The **lower the weight**, the **slower the vruntime grows**, giving higher priority.

♦ Example Scenario

- **CFS handles priority using vruntime scaling.**
- High-priority processes (like P2) run **longer** per unit time (vruntime increases slower).
- Low-priority processes (like P3) run **less** (vruntime increases faster).
- This ensures **weighted fairness**, not strict priority.

Example with priority (Nice Value)

Process Arrival Burst Nice Value Weight (Simplified)

P1	0	10	0	1.0
P2	2	4	-5	0.5
P3	4	6	+5	2.0

We simulate CFS behavior:

- **Actual time unit = 1**
- **vruntime increases per time unit:**
 - P1 $\rightarrow +1$
 - P2 $\rightarrow +0.5$
 - P3 $\rightarrow +2$

◇ Step-by-Step Execution with vruntime Tracking

Time	New Process?	Ready Queue	vruntime (P1,P2,P3)	Who Runs	Remarks
0	P1 arrives	{P1}	P1 = 0	P1	Only P1 is available
1		{P1}	P1 = 1	P1	
2	P2 arrives	{P1, P2}	P1 = 2, P2 = 0	P2	P2 has smaller vruntime
3		{P1, P2}	P1 = 2, P2 = 0.5	P2	P2 again
4	P3 arrives	{P1, P2, P3}	P1 = 2, P2 = 1.0, P3 = 0	P3	P3 lowest vruntime

Time	New Process?	Ready Queue	vruntime (P1,P2,P3)	Who Runs	Remarks
5		{P1, P2, P3}	P1 = 2, P2 = 1.0, P3 = 2	P2	P2 < P1, P3 (P2 again)
6		{P1, P2, P3}	P1 = 2, P2 = 1.5, P3 = 2	P2	Last burst for P2 (completes)
7		{P1, P3}	P1 = 2, P3 = 2	P1	Equal vruntime; choose P1
8		{P1, P3}	P1 = 3, P3 = 2	P3	P3 has smaller vruntime
9		{P1, P3}	P1 = 3, P3 = 4	P1	
10		{P1, P3}	P1 = 4, P3 = 4	P1	
11		{P1, P3}	P1 = 5, P3 = 4	P3	
12		{P1, P3}	P1 = 5, P3 = 6	P1	
13		{P1, P3}	P1 = 6, P3 = 6	P1	
14		{P1, P3}	P1 = 7, P3 = 6	P3	
15		{P1, P3}	P1 = 7, P3 = 8	P1	
16		{P1}	P1 = 8	P1	P3 finished now
17		{P1}	P1 = 9	P1	
18		{P1}	P1 = 10	P1	P1 done

Assignment III – Submit at department on Sunday,11 shrawon

1. Explain TSL(Test and Set Lock) and Peterson's algorithm.
2. What is TLB(Translation Lookaside Buffer) and Thrashing? Write Differences between Coalescing and Compaction with example.
3. What do you mean by asymmetric cryptosystem? Explain Caesar Cipher with suitable example.
4. Compare between system policies and access control on security implementation. How does ACL Work. Illustrate